

How attention works

There is a trained ‘value’ map from the space $\mathbb{R}^{512}$ containing the words (which are not linearly independent) to a lower dimensional ‘value space’ like $\mathbb{R}^{10}$.

$$R^{512} \rightarrow R^{10}$$

Each word has a trained ‘key’ vector and a trained ‘query vector’ (also with few entries like 5 entries).

When a sentence includes ‘car’ you dot product the query vector with the key of each other word (including ‘car’ itself) so you might find the key of ‘red’ has a high dot product with the query of ‘car.’ But the key of ‘car’ probably has a higher dot product with the query of ‘car’ as ‘being the actual word’ matters. You apply softmax and get a sequence of coefficients adding to 1. The corresponding linear combination of the value vectors is an element in value space $\mathbb{R}^{10}$. In many cases the softmax will have just one coefficient of 1 and the rest 0, the linear combination will just be the trained value vector of ‘car’.

There is a trained map the other way from value space to model space $\mathbb{R}^{10} \rightarrow \mathbb{R}^{512}$. The linear combinations of values with coefficients coming from dotting the query vector of ‘car’ with all the key vectors and taking softmax so the coefficients add to 1, when that linear combination which is an element of value space $\mathbb{R}^{10}$ is mapped back to $\mathbb{R}^{512}$ it is added to the word ‘car’ to adjust it.

In this way, using ordinary linear algebra, the sequence of vectors representing the words in a sentence is adjusted to get a new sequence of elements of R^{512} which is just another sequence of the same length, but it better encodes the meaning.

Positional encoding is a similar type of adjustment which instead adjusts the vector associated to each word in a sentence in such a way that the vectors corresponding to the words are moved a bit to make later linear algebra be able to access which word comes before which other if it needs to do that. For example you add the sin of i times the position of each word to its $2i$ coordinate in $\mathbb{R}^{512}$ and

you add i times the cosine of the position of the word to its $2i + 1$ coordinate.

Both attention and position encoding are pure linear algebra steps that take a sequence of elements of $\mathbb{R}^{512}$ and output another sequence of elements of $\mathbb{R}^{512}$ of the same length, with each vector having been adjusted by adding to it extra vectors according to ways it is affected by other nearby words.

After a subsequent ReLu step, we again get a sequence of elements of $\mathbb{R}^{512}$ but these no longer correspond to words at all, yet we proceed on as if they were, doing another attention step, no position encoding step now I think, and another ReLu etc.